# Optimal Wiring of Heliostats in Solar Tower Power Plants

Lab-course Computer Science
Summer semester 2018

Duc Thanh Tran

09 July 2018

# Motivation

Figure: Solar tower power plant. Heliostats reflect onto a receiver in order to heat up a medium for energy conversion.

Sun's position changes continuously, thus for effective energy conversion we need to rotate the heliostats.

$\Rightarrow$ each heliostat requires a data and power cable

# Problem

- Consider a complete and undirected graph $G = (V, E)$ with $V := \{T\} \cup V_H$, where $T$ is the designated solar tower and $V_H$ is the set of heliostats.

# Problem

- Consider a complete and undirected graph $G = (V, E)$ with $V := \{T\} \cup V_H$, where $T$ is the designated solar tower and $V_H$ is the set of heliostats.
- Goal: Find subgraphs $H_{\text{data}}, H_{\text{power}} \subseteq G$ with minimal total costs that both connect all heliostats with the solar tower.
  - data cable: cable and switch costs
  - power cable: cable costs

# Problem

- Consider a complete and undirected graph $G = (V, E)$ with $V := \{T\} \cup V_H$, where $T$ is the designated solar tower and $V_H$ is the set of heliostats.
- Goal: Find subgraphs $H_{\text{data}}, H_{\text{power}} \subseteq G$ with minimal total costs that both connect all heliostats with the solar tower.
  - data cable: cable and switch costs
  - power cable: cable costs
- Constraints:
  - no intersecting edges
  - cables have capacity and length constraints
  - switches limit connectivity at heliostats for data cables

# Problem: Data cable

Table: Data cables with their limitations and costs.

| Cable | Max. capacity | Max. length [m] | Price [€/m] |
|---|---|---|---|
| Copper | 2 | 100 | 0.7 |
| Fiberglass | - | - | 2 |

- additionally: protective foil (+2€/m)

# Problem: Data cable

Table: Data cables with their limitations and costs.

| Cable | Max. capacity | Max. length [m] | Price [€/m] |
|---|---|---|---|
| Copper | 2 | 100 | 0.7 |
| Fiberglass | - | - | 2 |

- additionally: protective foil (+2€/m)

Table: Switch prices, for 8-/16-port switches we can either use copper or fiber-glass output. Endpoints are only used for copper cables.

| Switch | Max. degree | Price [€/p. piece] |
|---|---|---|
| Endpoint | 1 | 10 |
| Conductor | 2 | 100 |
| 8-port copper/fiberglass switch | 9 | 800 |
| 16-port copper/fiberglass switch | 17 | 1500 |

# Problem: Power cable

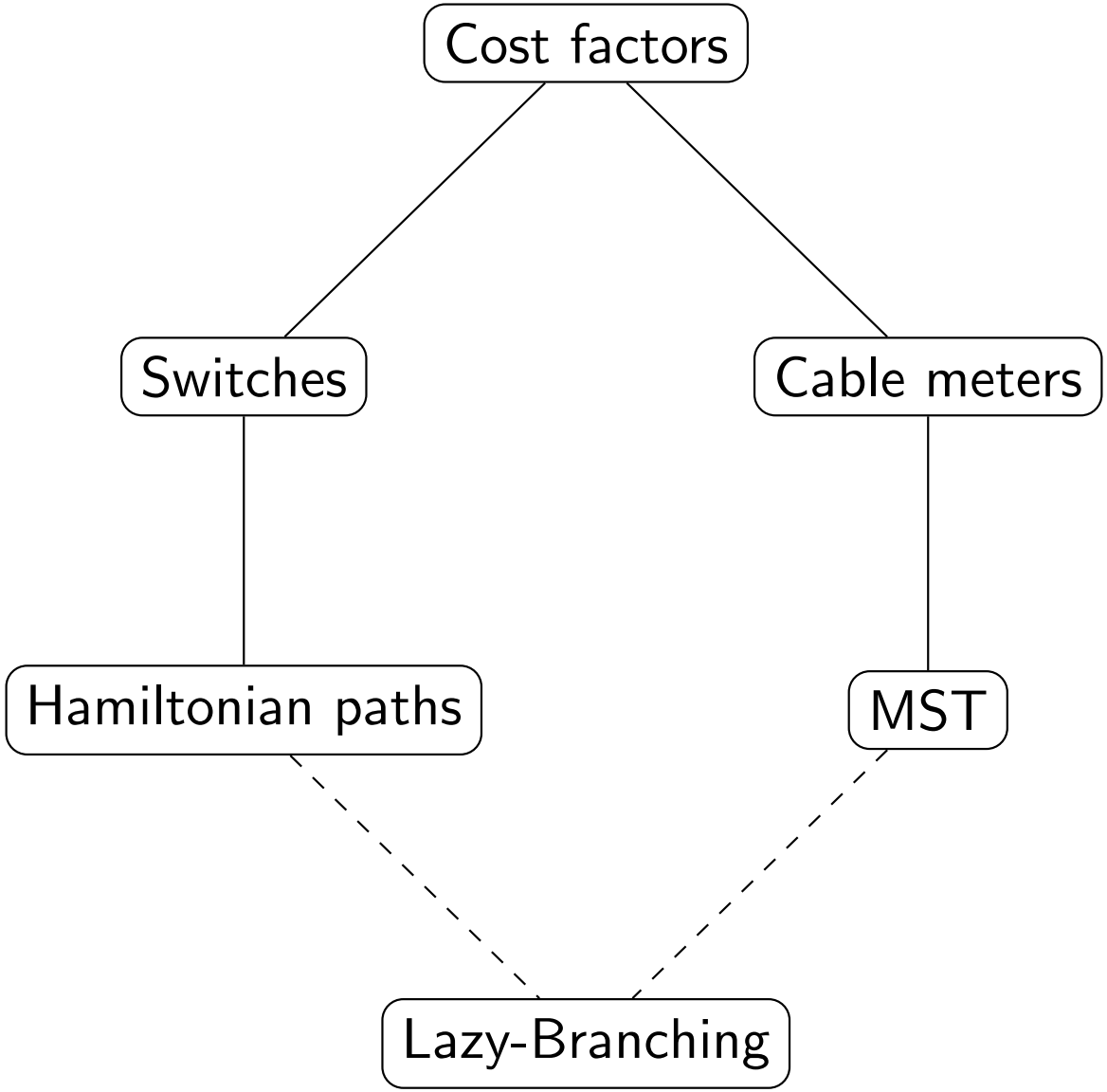
Table: Power cables with their limitations and prices.

| Cable | Max. capacity | Max. length [m] | Price [€/m] |
|---|---|---|---|
| NYY-J 3x2.5 RE | 56 | 38.36 | 0.58 |
| NYY-J 3x4 RE | 73 | 47.08 | 0.87 |
| NYY-J 3x6 RE | 92 | 56.04 | 1.24 |
| NYY-J 3x10 RE | 124 | 69.3 | 1.95 |
| NYY-J 3x16 RE | 162 | 84.87 | 3.13 |
| NYY-J 3x25 RM | 209 | 102.79 | 5.19 |
| NYY-J 3x35 RM | 250 | 120.31 | 6.90 |

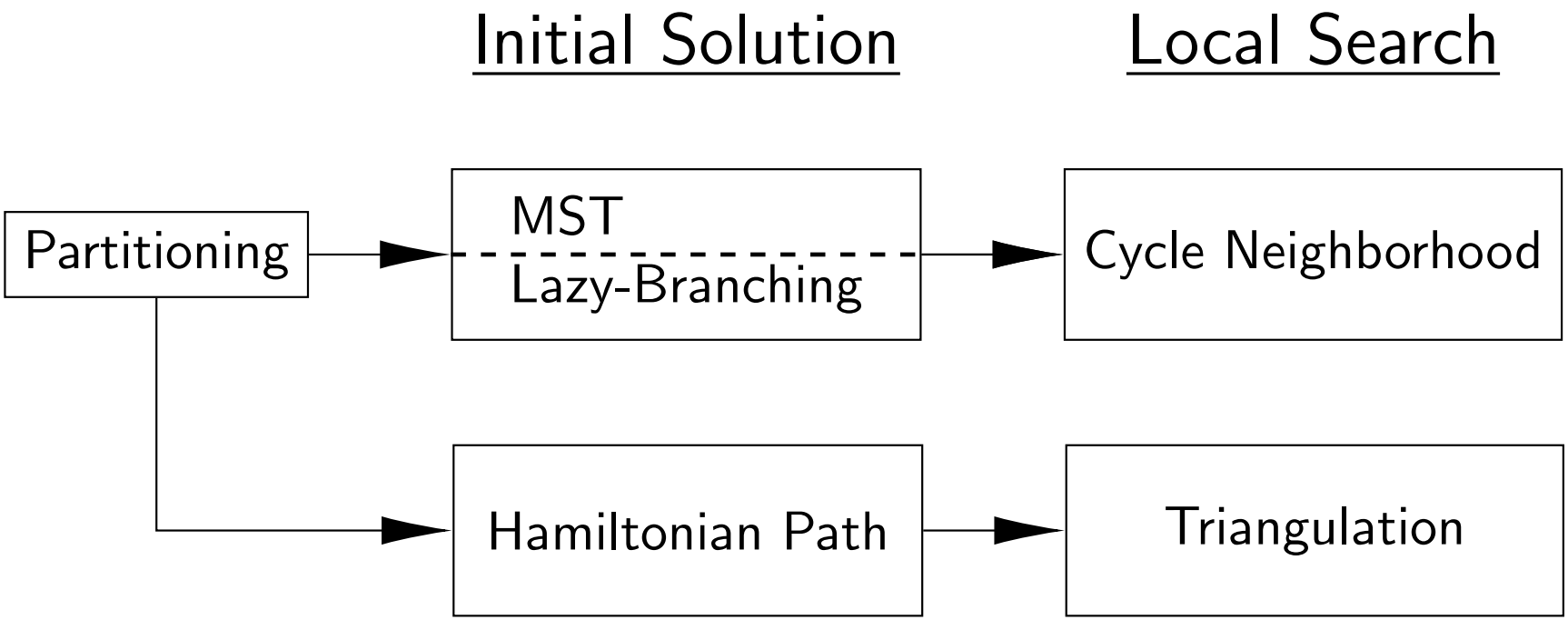
# Challenges

- How are heliostats partitioned?
- Data cable: use expensive switches that allow branching or consider paths
- How does the location affect solutions?

# Initial solutions



# Our approach



# Partitioning

Maximum number of connected heliostats per cable is limited by $h_{max} = 128$.

- Partition heliostats beforehand: Calculate angles between vectors and the x-axis.
- Compute initial solutions on each partition independently.

# Partitioning

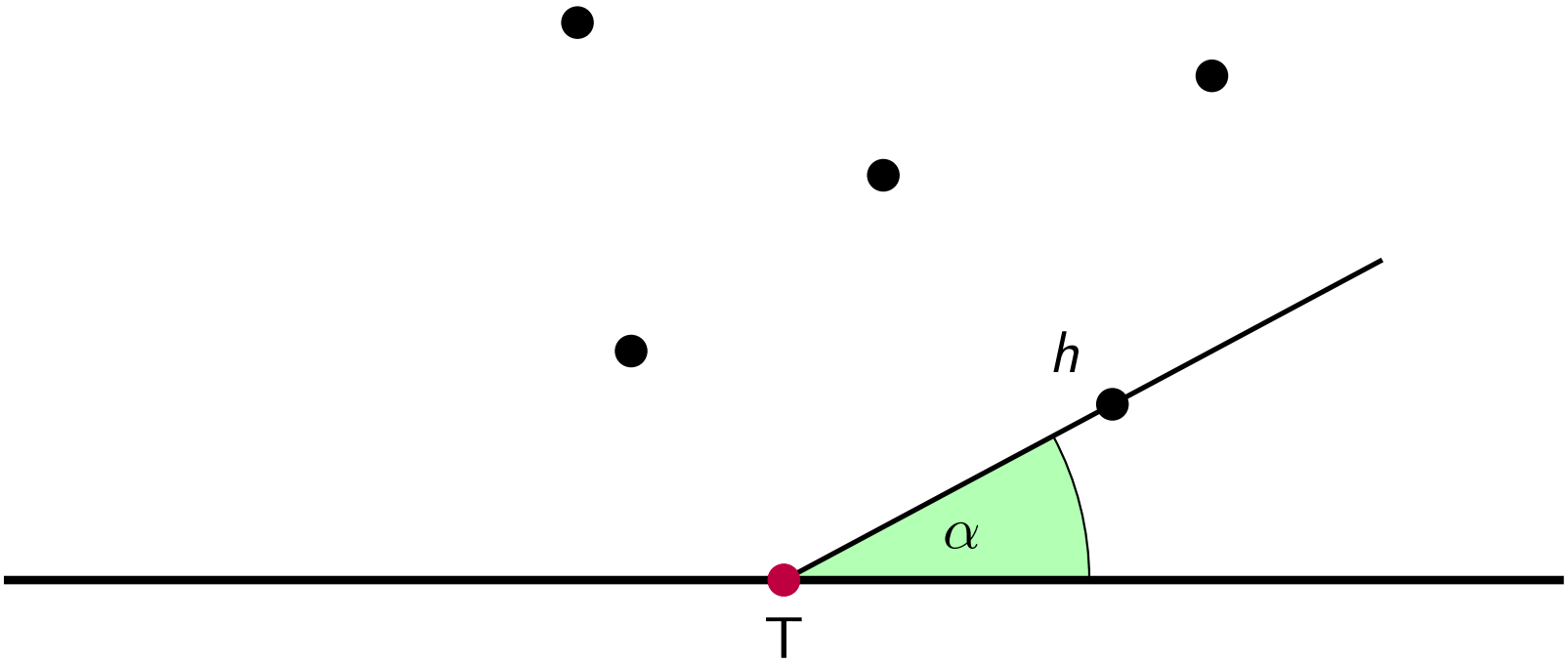



Figure: Compute angle between x-axis and $line(T, h)$, $h \in V_H$.

# Partitioning

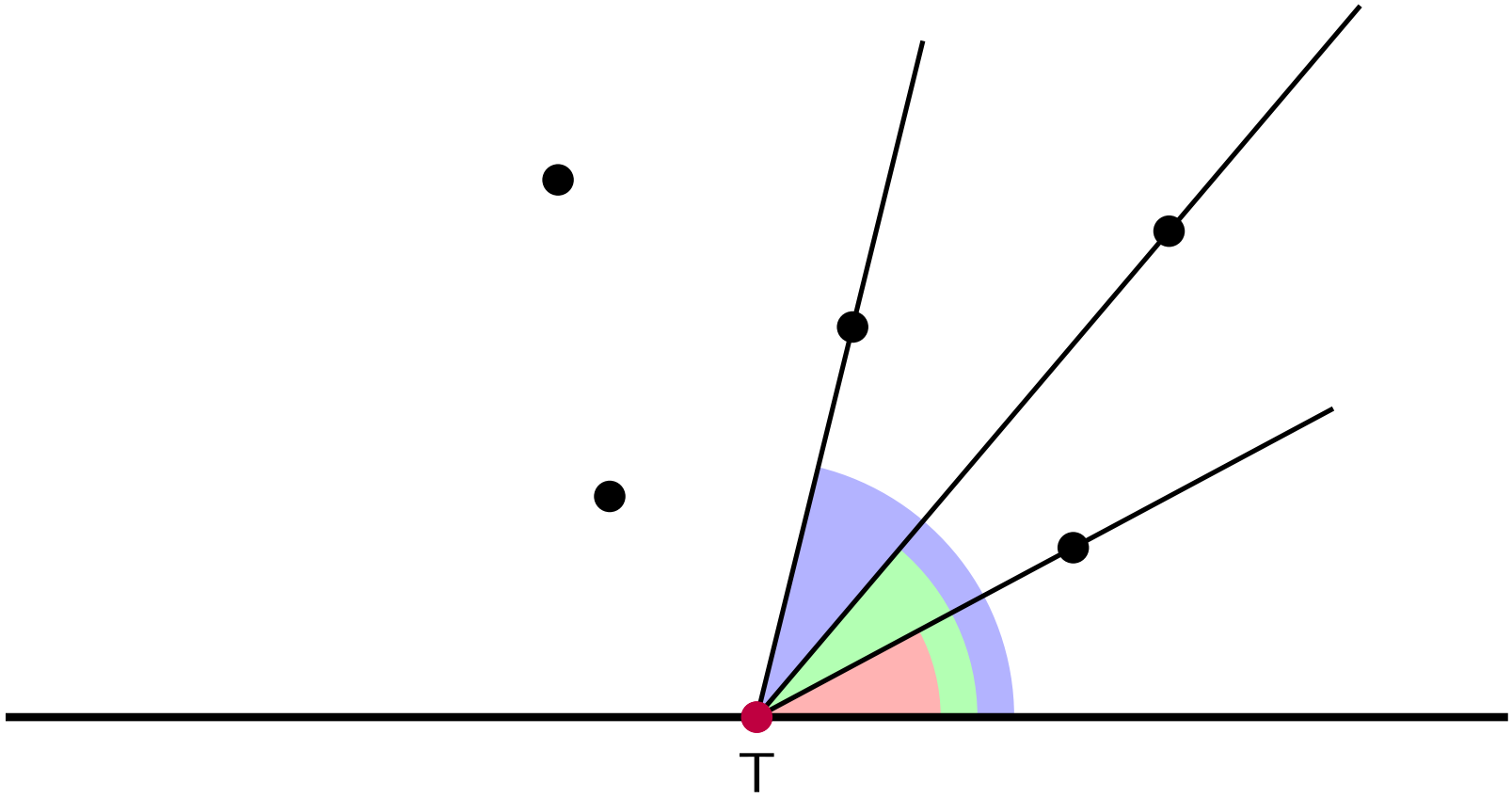
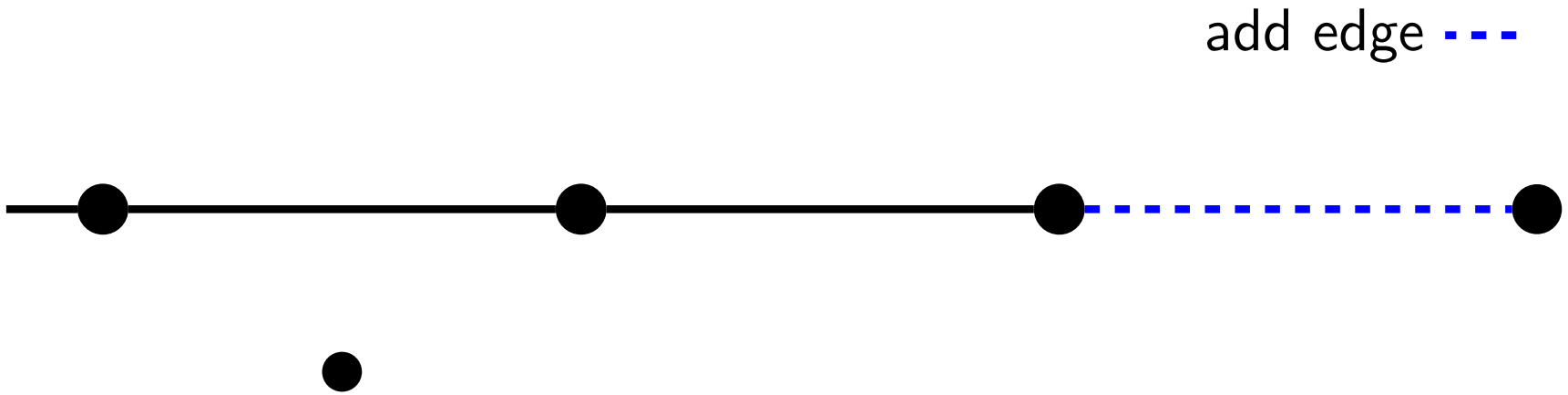



Figure: Sort heliostats by computed angles.

# Heuristic: Hamiltonian Path

# Initial Solution: Hamiltonian Path

i) Append




ii) Insert

# Local Search: Hamiltonian Path – Triangulation

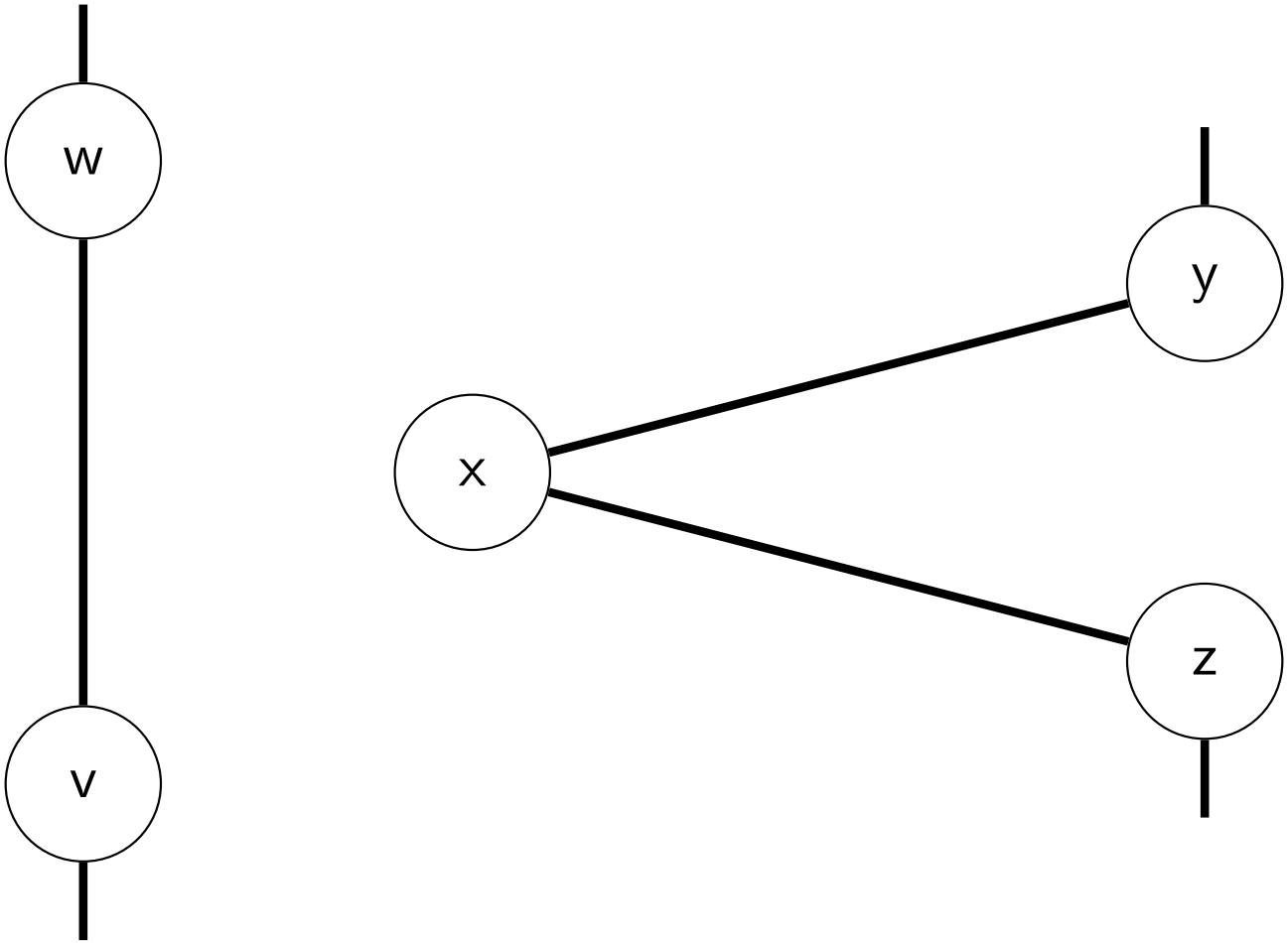



Figure: Starting point.

# Local Search: Hamiltonian Path – Triangulation

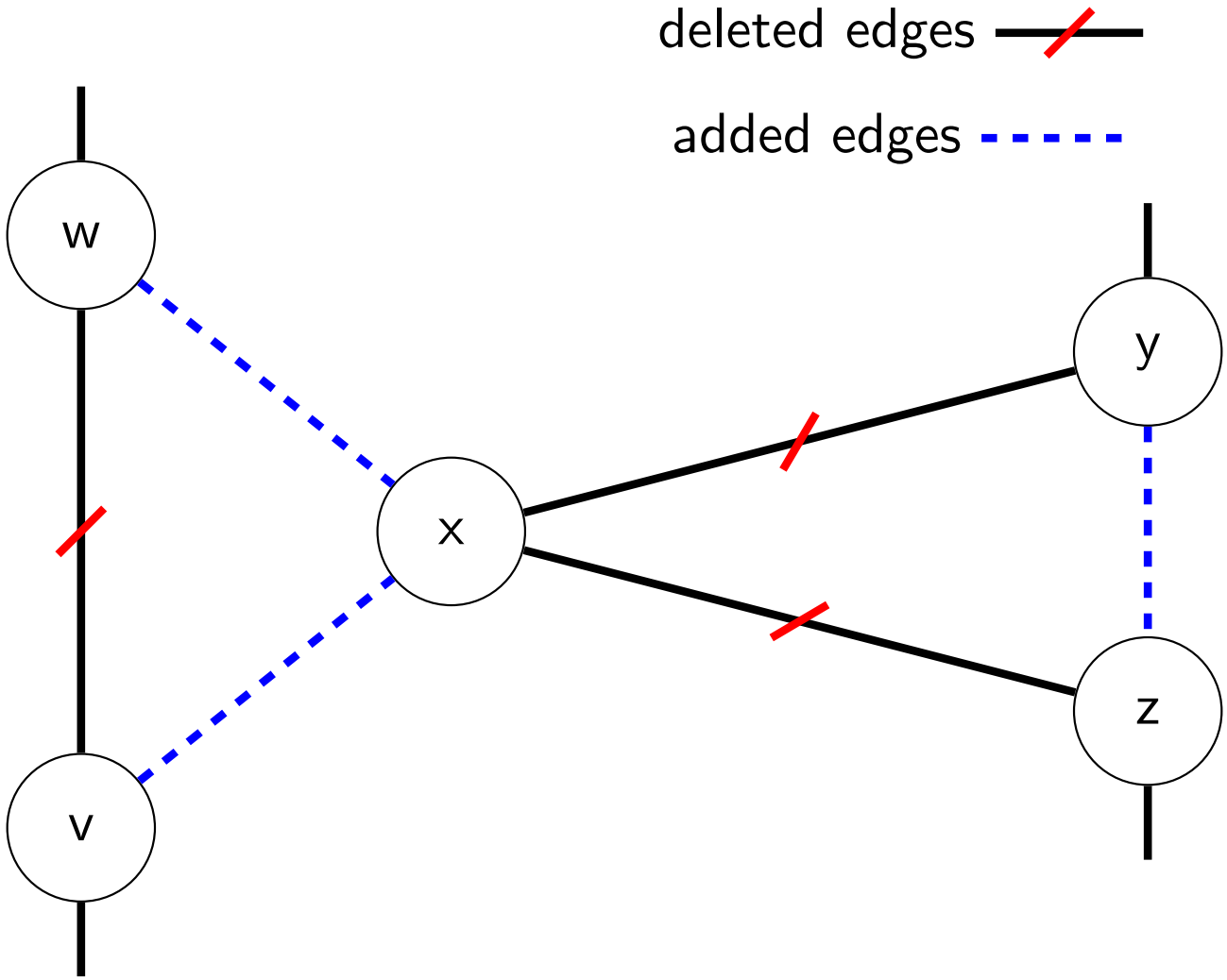



Figure: Check costs of added and deleted edges.

# Local Search: Hamiltonian Path – Triangulation

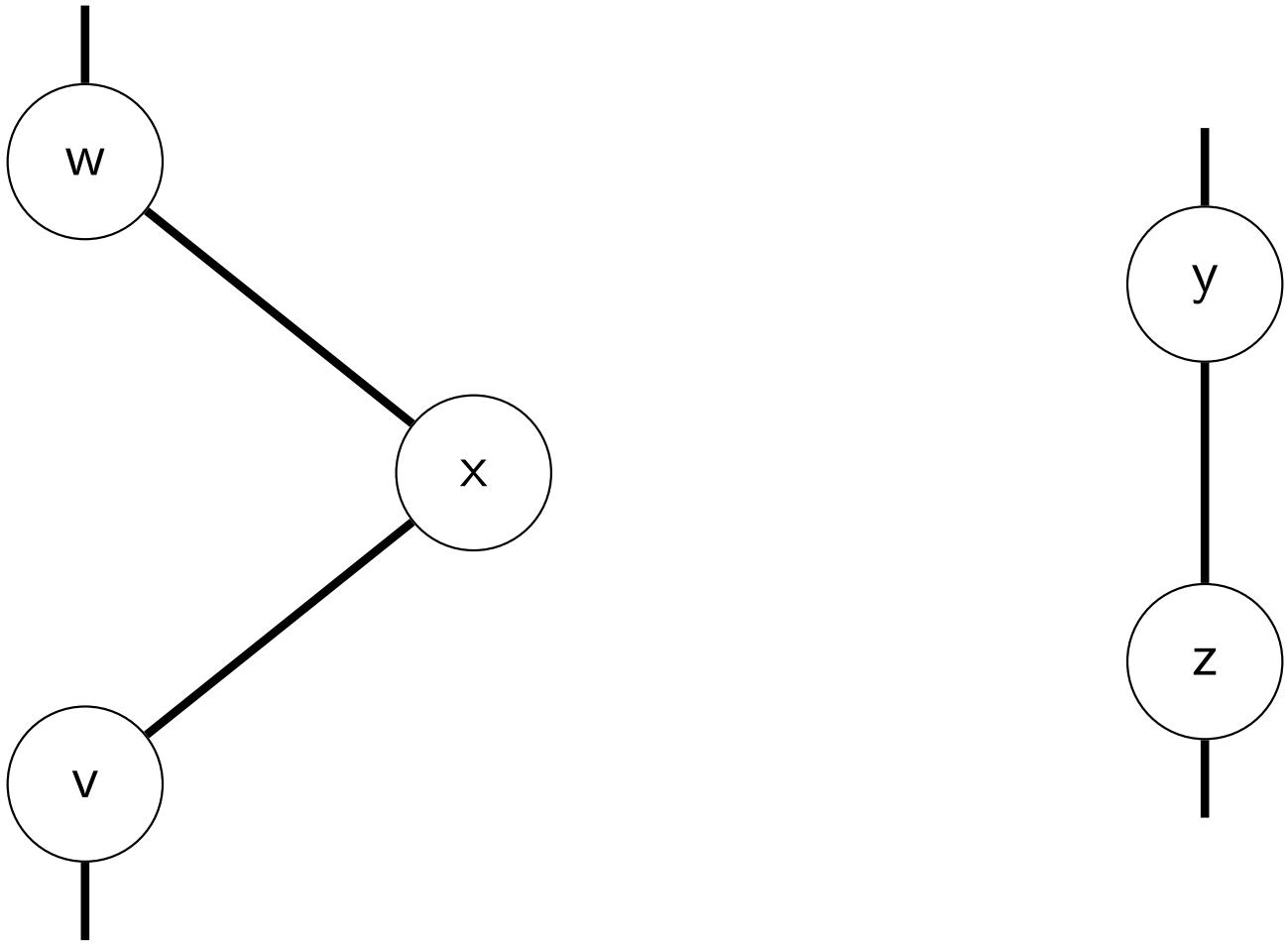
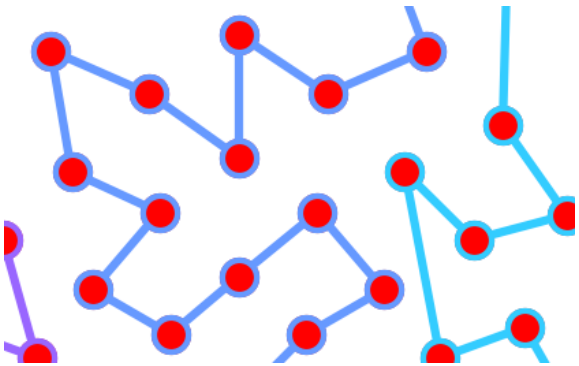
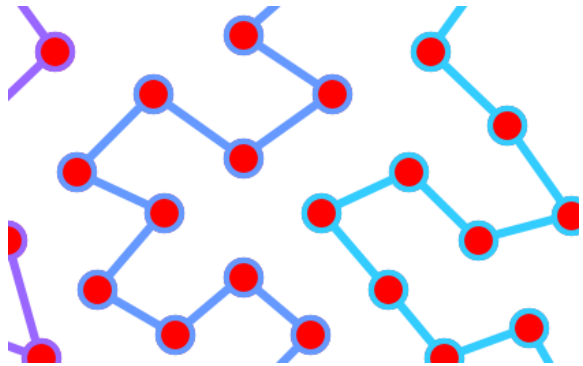



Figure: Added edges have lower costs than deleted ones.

# Triangulation Neighborhood Improvement



(a) Initial solution.



(b) After triangulation improvement.

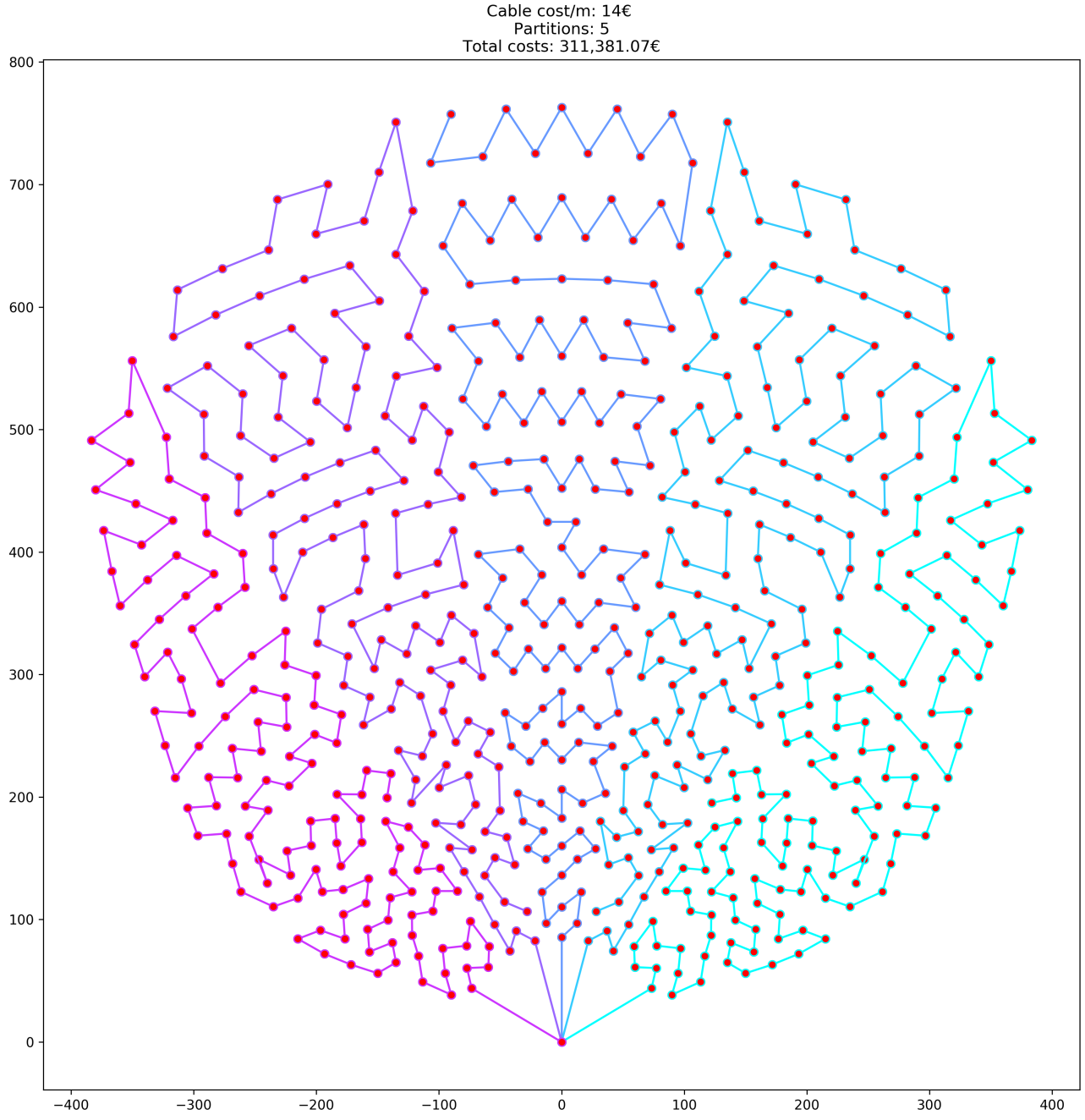



Figure: Initial Hamiltonian path. Total costs: 311,381.07€.

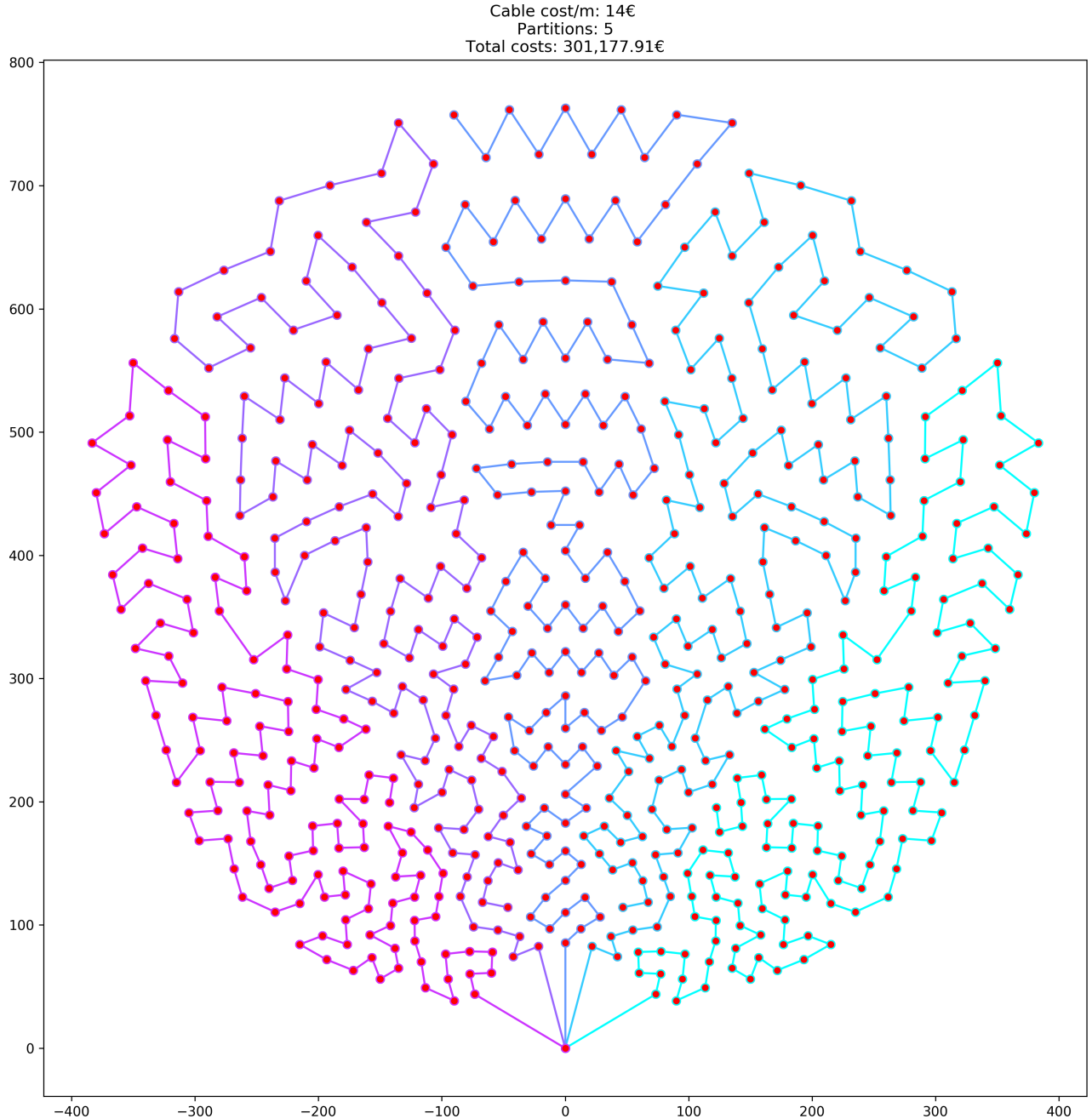



Figure: After local search for Hamiltonian paths. Total costs: 301,177.91€.

# Conclusion

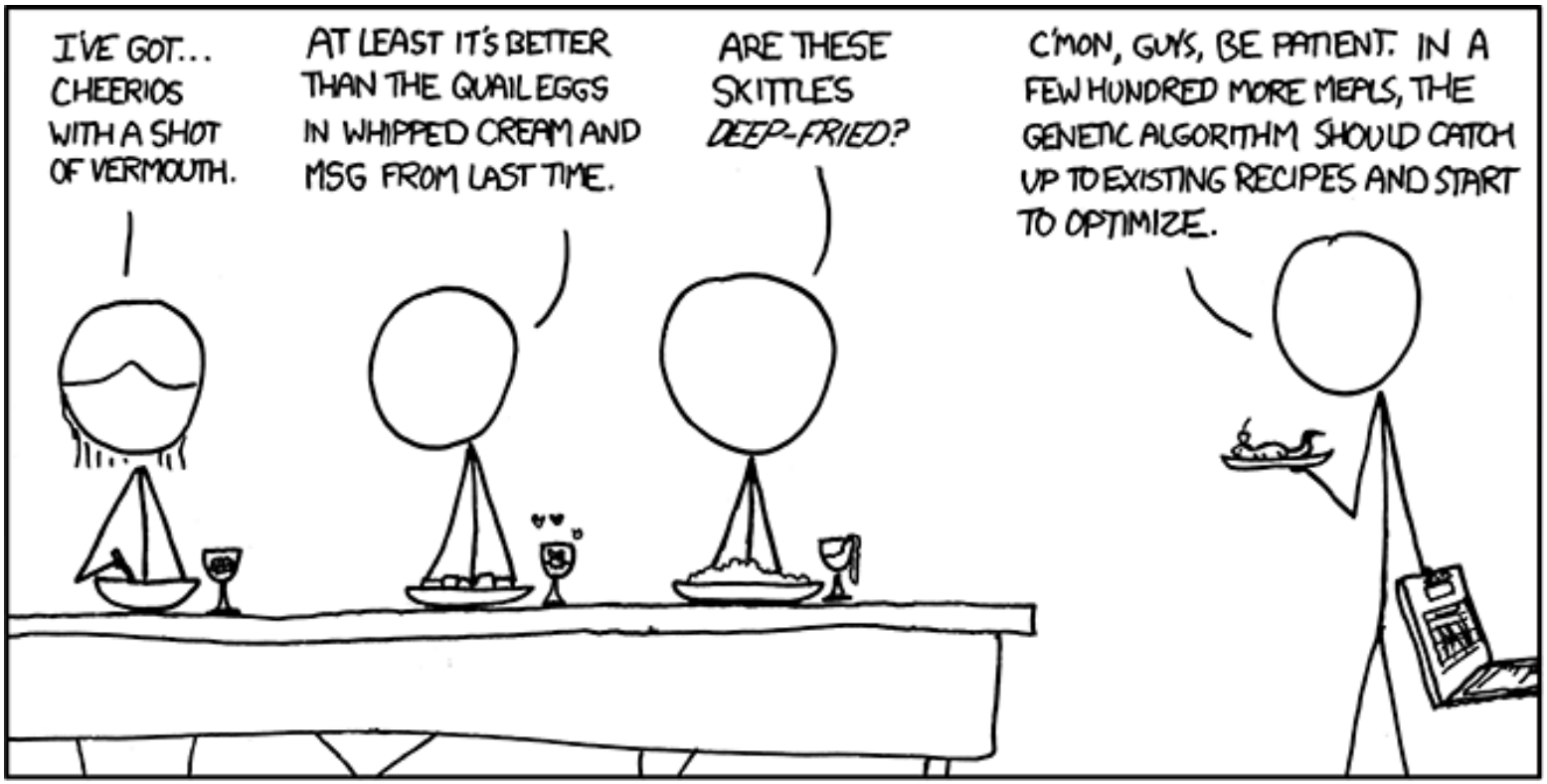
**Unanimous result**

# Conclusion

**Unanimous result**

$\Downarrow$

**Hamiltonian path with 5 tours delivers best results**




**Thank you for your attention!**

# Appendix: Algorithms

# Initial Solution: Hamiltonian Path

**Algorithm 1** Nearest-insertion heuristic for Hamiltonian path construction.

1: **function** NEAREST-INSERTION-HAMILTONIAN($G = (V, E), c$)
2: $Seq := [T]$ ▷ Ordered sequence of vertices
3: **while** $|Seq| < |V|$ **do**
4: $(v, index) :=$ BEST-CANDIDATE($Seq, V, c$)
5: $Seq[index] := v$
6: **end while**
7: construct corresponding edges from $Seq$, yielding $E'$
8: **return** (V,E')
9: **end function**

# Local Search: Triangulation for Hamiltonian Path

**Algorithm 2** Local Search: Triangulation

```
1:  function LOCAL-SEARCH-TRIANGULATION(G = (V, E), c)
2:      while not stable do                          ▷ No edge can be improved
3:          for vw ∈ E do
4:              IMPROVE-EDGE(vw, G, c)
5:          end for
6:      end while
7:      return G = (V, E)
8:  end function
9:
10: function IMPROVE-EDGE(vw, G, c)
11:     (xy, xz) := GET-BEST-TRIANGULATION(vw, G)
12:      (xz may be NULL)
13:     if HAS-NEGATIVE-COSTS(vw, x, G, c) then
14:         E := E \ {vw, xy, xz}
15:         E := E ∪ {vx, wx} ∪ {yz}          ▷ last edge set can be empty
16:     end if
17:     return
18: end function
```

# Initial Solution: Minimum Spanning Tree

**Algorithm 3** Prim's Algorithm with additional intersecting-constraint.

1: **function** MST-PRIM($G = (V, E)$, $c$)
2: $E' := \emptyset$
3: **while** $|E'| < |V|$ **do**
4: $F :=$ SORTED-CUT-EDGES($E', c$) ▷ Consider edges $Tw \in E$ if $E' = \emptyset$
5: **for** $e \in F$ **do**
6: **if** NOT-INTERSECTING($e, E'$) **then**
7: $E' := E' \cup e$
8: **go to** 3
9: **end if**
10: **end for**
11: **end while**
12: **return** (V,E')
13: **end function**

# Initial Solution: Lazy-branching

**Algorithm 4** Hybrid algorithm that computes a spanning tree by extending paths greedily as possible. Otherwise we branch off and install switches.

1: **function** LAZY-BRANCHING($G = (V, E)$, $c$)
2: $last := T$, mark $T$ as visited ▷ Start from solar tower
3: $E' := \emptyset$
4: **while** $|E'| < |V|$ **do**
5: select unvisited vertex $w \in V$ such that $c_e$ is minimized for $e := \{last, w\} \in E$
6: **if** NOT-INTERSECTING($e, E'$) **then** ▷ Path
7: $E' := E' \cup e$
8: $last := w$, mark $w$ as visited
9: **else** ▷ Branching
10: select visited vertex $v \in V$ such that $c_f$ is minimized for $f := \{v, w\} \in E$
11: $E' := E' \cup f$
12: $last := w$, mark $w$ as visited
13: **end if**
14: **end while**
15: **return** (V,E')
16: **end function**

# Local Search: Cycle Neighborhood

**Algorithm 5** Local search: Cycle Neighborhood for Spanning Trees

```
1: function LOCAL-SEARCH-CYCLE-NEIGHBORHOOD(G = (V, E), c)
2:     while not stable do                          ▷ No edge can be improved
3:         for e ∈ E do
4:             IMPROVE-EDGE(e, G, c)
5:         end for
6:     end while
7:     return G = (V, E)
8: end function
9:
10: function IMPROVE-EDGE(e, G, c)
11:     E := E \ {e} ⇒ obtain disjoint partitions V1, V2 ⊆ V
12:     for e ∈ SORTED-CUT-EDGES(V1, V2, c) do
13:         if NOT-INTERSECTING(e, E) then
14:             E := E ∪ {e}
15:             return
16:         end if
17:     end for
18:     E := E ∪ {e}                          ▷ No improvement by replacing e
19:     return
20: end function
```